

Recursion

1. What is Recursion
2. Example of Recursion
3. Tracing Recursion
4. Stack used in Recursion
5. Time complexity
6. Recurrence Relation

1. What is Recursion

Type Fun(Param)

if (< base condition >) → There must be

same condition

2. Fun(Param) → to stop recursion otherwise it will

Keep on calling

2. Example of Recursion & Tracing

void Fun1(int n)

{ if (n > 0)

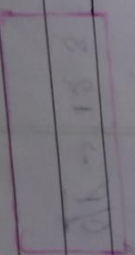
{ printf("%d", n);

2. Fun1(n-1);

void main()

{ int x = 5;

Fun1(x);



Tracing →

it goes to main

Fun 1 (3) → it goes back to previous & comes out of 3

it goes back to previous & comes out of 3

Q/P → 3 2 1

Fun 1 (1) → it goes back to previous & comes out of 1

come out of 1

void fun2(int n)

Firstly
Fun2(3)

if (n > 0)
1. Fun2(n-1);
2. Print ("id", n);

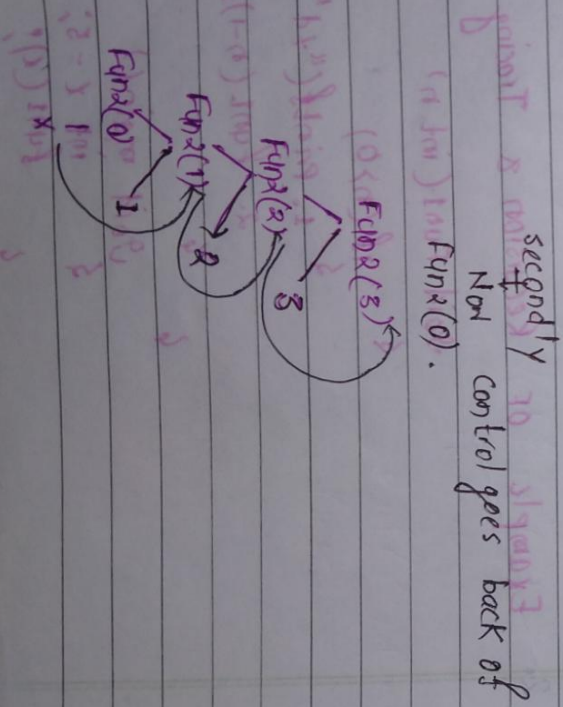
Fun2(0)

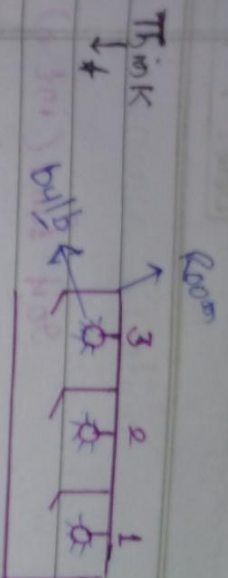
void main()

int x = 3;

fun2(x);

Q/P → 1 2 3





1 switch on Bulb

like recursive → 2. go to next Room

O/P: 321

→ After room it will return without doing anything

like recursive → 1. go to next Room

2. switch on Bulb

O/P: 123

Generalising Recursion

void fun (int n)

if (n > 0) {
 // do something
 fun(n-1);
}

Ascending 1 Calling

2 fun(n-1) * 2

Descending 3. Returning

if (n > 0) {

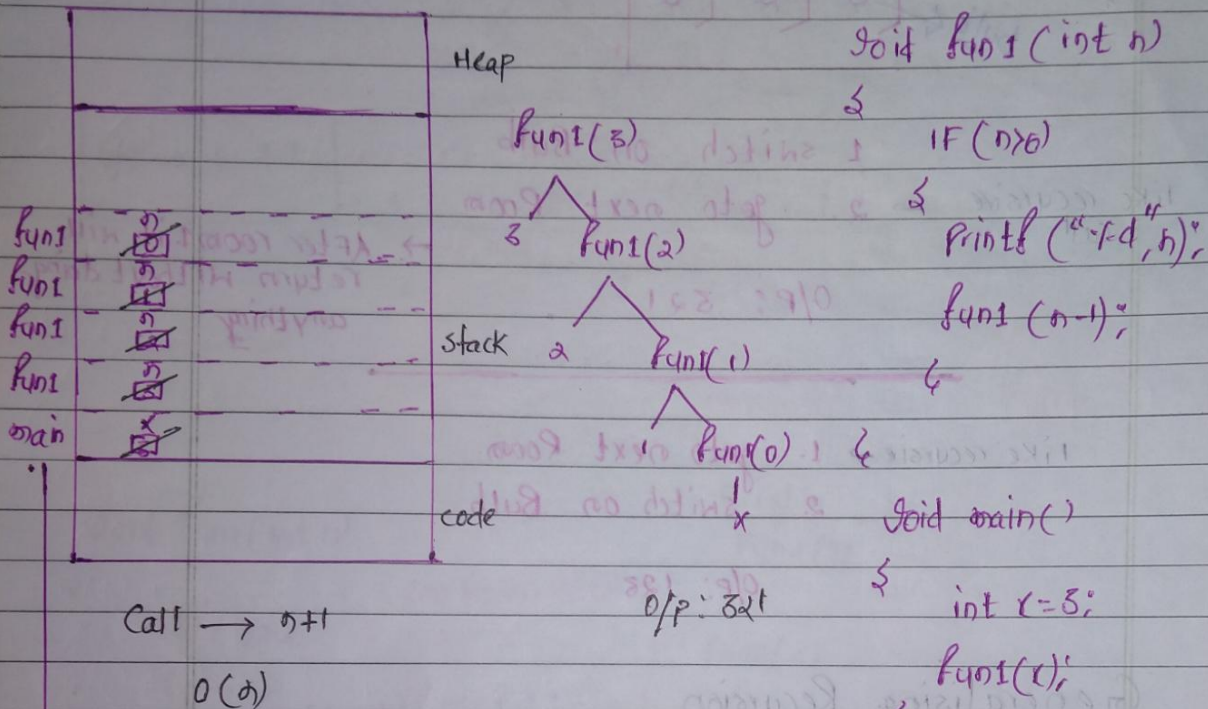
 fun(n-1);

(1) Ascending

Recursion has two phases: (2) Descending

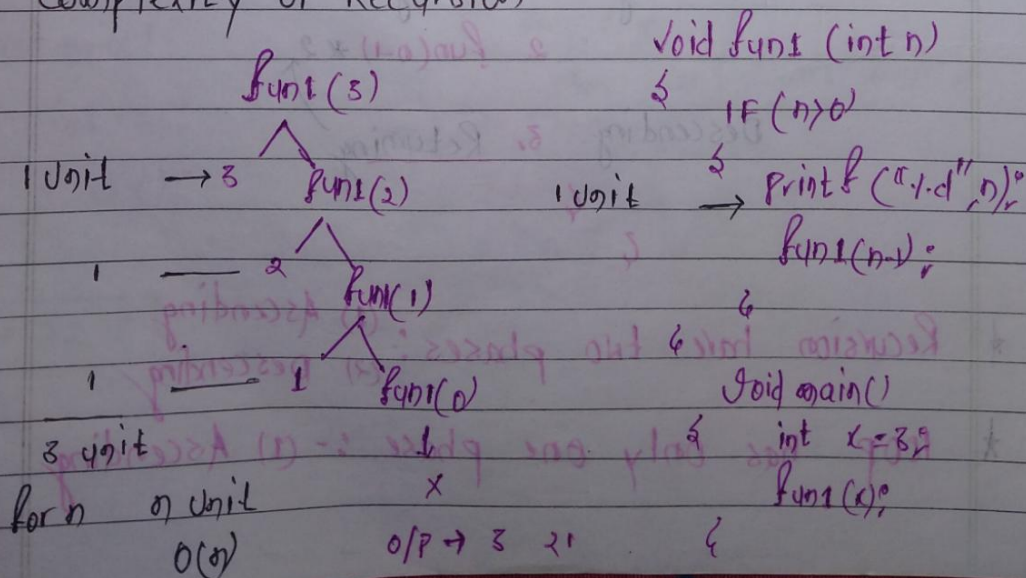
* loop has only one phase :- (1) Ascending

How Recursion uses stack



→ At each call there is new activation record is being created for func()

Time Complexity of Recursion



Recurrence Relation

* Let total time in this function is $T(n)$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1)+2 & n>0 \end{cases}$$

↓
Recurrence Relation

$T(n)$ — void fun1 (int n)

{ if (n>0)

{ printf("%d", n);

fun1(n-1);

$T(n) = T(n-1) + 2$

Solving Above relation :-

replace 2 by 1 as it is const

$$T(n) = T(n-1) + 1 \quad \text{--- (1)}$$

$$\therefore T(n) = T(n-1) + 1$$

$$\therefore T(n-1) = T(n-2) + 1$$

$$T(n) = T(n-2) + 1 + 1$$

$$T(n) = \frac{T(n-2) + 2}{T(n-3) + 1}$$

$$T(n) = T(n-3) + 1 + 2$$

$$T(n) = T(n-3) + 3$$

⋮

$$T(n) = T(n-k) + k \quad \text{--- (4)}$$

(As we know answer at $n=0$)
Assume $n-k=0 \therefore n=k$

$$T(n) = T(n-1) + n$$

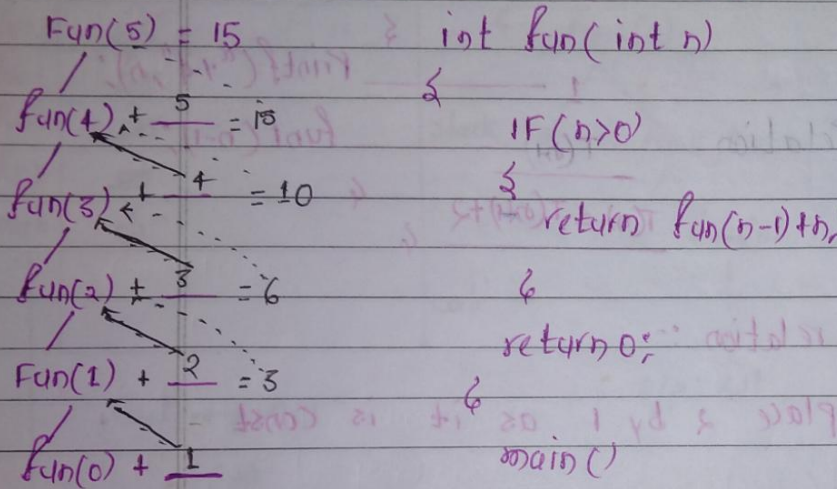
$$T(n) = T(0) + n$$

$$T(n) = 1 + n \quad \text{--- } O(n)$$

** very Useful in recursion

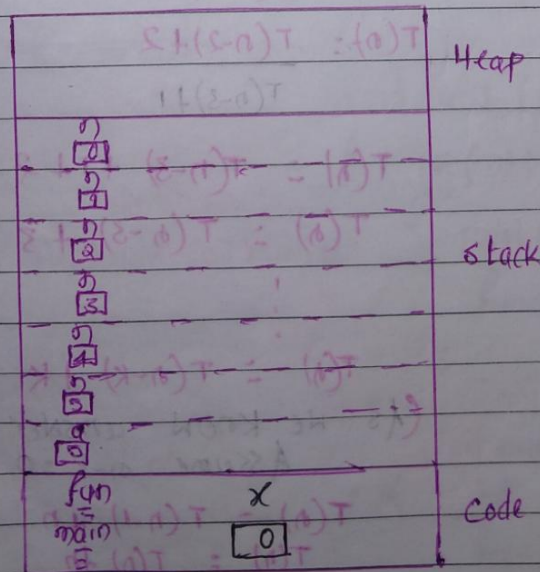
Static and global variable in Recursion

1. Static variable in Recursion



all addition will be done during returning time

★ static variable are created only once



* global variable

During Calling $\rightarrow$ $\star\star$ created only once (static variable)

Fun(5)
/ Fun(4)
/ Fun(3)
/ Fun(2)
/ Fun(1)
/ Fun(0)

$\boxed{0} + 2 + 3 + 4 + 5$

int fun(int n)

{ static int x=0;

if (n>0)

{ x++;

return fun(n-1) + x;

}
return 0;

int x=0

int fun()

{

↓

same result

will come

}

During returning

Fun(5) = 25

20
Fun(4) + 5 = 25

15
Fun(3) + 5 = 20

10
Fun(2) + 5 = 15

5
Fun(1) + 5 = 10

0
Fun(0) + 5 = 5

0

main()

{ int a=5;

printf("%d", fun(a));

↓
25

* static & global variable created inside code section

Types of Recursion

1. Tail Recursion
2. Head Recursion
3. Tree Recursion
4. Indirect Recursion
5. Nested Recursion

1. Tail Recursion

means, At returning time it doesn't have to perform anything at all.

Fun(n)

```
{
  IF(n > 0)
```

```
{
  ==
  ==
```

```
  fun(n+1) + n;
```

```
}
```

↓

* not tail recursion, as

it has to perform +n during returning

Void Fun(int n)

```
{
```

```
  IF(n > 0)
```

```
  {
    printf("%d", n);
```

```
    fun(n-1);
```

```
}
```

Tail recursion ←

```
{
```

```
  fun(3);
```

* Every Recursion can be written in loop & vice versa

⇒ Comparing Tail recursion with loop

Void Fun(int n)

```
{
```

```
  While(n > 0)
```

```
  {
    printf("%d", n);
```

```
    n--;
```

```
}
```

Time - $O(n)$

* Space - $O(1)$

Void fun(int n)

```
{
```

```
  IF(n > 0)
```

```
  {
    printf("%d", n);
```

```
    fun(n-1);
```

```
}
```

```
  fun(3);
```

Time - $O(n)$

Space - $O(n)$

* In case of Tail recursion, it is better you convert it into loop as loop is more efficient in term of space

* In some compiler, compiler tries to convert Tail recursion in loop for making efficient of space.

2. Head Recursion

↓
it perform all the thing after recursive call.

* It has to do everything during returning time
or Perform

void Fun(int n)

```

{
    IF (n > 0)
    {
        Fun(n-1)
        //
        //
        //
    }
}

```

void Fun(int n)

```

{
    IF (n > 0)
    {
        Fun(n-1)
        printf("%d", n)
    }
}

```

Head Recursion vs loop

void Fun(int n)

```

{
    int i = 1;
    while (i <= n)
    {
        printf("%d", i)
        i++
    }
}

```

O/P :- 1, 2, 3

void Fun(int n)

```

{
    IF (n > 0)
    {
        Fun(n-1)
        printf("%d", n)
    }
}

```

Fun(3)

O/P :- 1, 2, 3

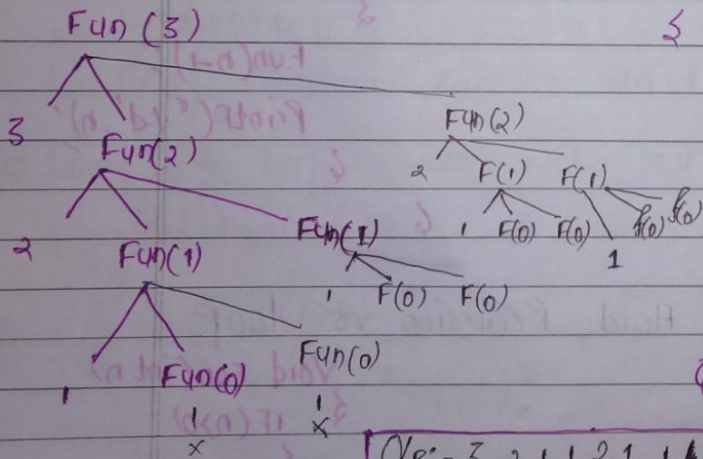
* Head Recursion can't easily be converted in loop, but it can be converted but it doesn't look same like

Linear Recursion

```

Fun(n)
{
    IF (n > 0)
    {
        Fun(n-1);
    }
}
    
```

→ visualise



Tree Recursion

IF a recursive Function calling itself more than one time

```

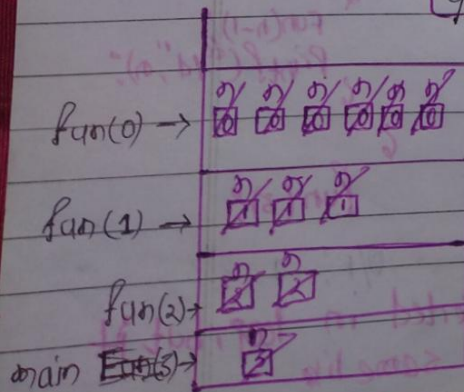
Fun(n)
{
    IF (n > 0)
    {
        Fun(n-1);
        Fun(n-1);
    }
}
    
```

Void Fun(int n)

```

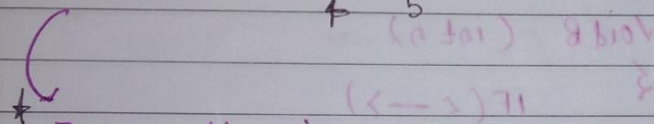
{
    IF (n > 0)
    {
        1. printf("%d", n);
        2. Fun(n-1);
        3. Fun(n-1);
    }
}
    
```

Q.P: - 3, 2, 1, 1, 2, 1, 1, 4



stack

Only Four activation record created For Four Function



For $n = 4$, there will be 5 layer

For $n = 4$, there will be 5 layer

For $n = 5$, 10^5

$$f(a) > 0$$

(19) 8007

1. $\frac{1}{2} \log \frac{1}{2}$

space -

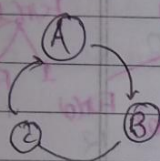
~~11. (45)~~

(10/10/17)

1. Proof (14)

$$f(x) = \frac{1}{x^2}$$

Indirect Recursion



void A (int n)

{
IF (< ->)

{
B(n-1);

void B (int n)

{
IF (< ->)

{
A(n-3);

FunA(20)

20

FunB(19)

19

FunA(9)

9

FunB(8)

8

FunA(4)

4

FunB(3)

3

FunA(1)

1

FunB(0)

X

void FunA (int n)

{
IF (n>0)

{
1. printf ("%d", n);

2. FunB (n-1);

{

void FunB (int n)

{
IF (n>1)

{
1. printf ("%d", n);

2. FunA (n/2);

{

Nested Recursion

↓
Parameter itself is a recursive call

void Fun (int n)

{
 if (—)

{
 =

Fun(Fun(n-1));

}

Fun(95) = 91

↓

Fun(Fun(95+11))

↕
Fun(96)

↓

Fun(Fun(107))

↕
Fun(97)

↓

Fun(Fun(108))

↕
Fun(98)

↓

Fun(Fun(109))

↕
Fun(99)

↓

Fun(Fun(110))

↕
Fun(100)

96 = Fun(106)

97 = Fun(107)

98 = Fun(108)

99 = Fun(109)

Fun(Fun(111))

↕
101

Fun(101)

↕
91

int Fun(int n)

{

 if (n > 100)

 return n - 10;

 else

 return Fun(Fun(n+11));

}

Fun(95)

Sum of First 'n' Natural Number

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + \dots + n$$

$$1 + 2 + 3 + 4 + \dots + (n-1) + n$$

$$\text{Sum}(n) = 1 + 2 + 3 + 4 + \dots + (n-1) + n$$

$$\text{Sum}(n) = \text{Sum}(n-1) + n$$

$$\text{Sum}(n) = \begin{cases} 0 & n = 0 \\ \text{Sum}(n-1) + n & n > 0 \end{cases}$$

★ In defining recurrence relation, you must have two things

- (i) For large value of n what will be FORMULA
- (ii) For small value of n what will be ANSWER

$$\begin{aligned} \text{Sum}(5) &= 15 \\ \text{Sum}(4) + 5 &= 15 \\ \text{Sum}(3) + 4 &= 10 \\ \text{Sum}(2) + 3 &= 6 \\ \text{Sum}(1) + 2 &= 3 \\ \text{Sum}(0) + 1 &= 1 \\ 0 \end{aligned}$$

```
int sum(int n)
```

```
if (n == 0)
    return 0;
```

```
else
```

```
return sum(n-1) + n;
```

↖ n+1 Call

Time $\rightarrow O(n)$

Space $\rightarrow n+1 \rightarrow O(n)$

* For problem solving, recursion are easy to use but it take more time & space as compared to loop. But in Applⁿ. We use loop

```
int sum(int n)
```

```
{
    return n*(n+1)/2;
}
```

```
int sum(int n)
```

```
{
    int i, s = 0; — ①
```

```
    For(i=1; i<=n; i++) = n+1
```

```
    s = s + i; — n
```

```
    return s; — 1
```

```
}
```

 $O(n)$

Factorial of a Number

$$n! = 1 * 2 * 3 * \dots * n$$

$$5! = 1 * 2 * 3 * 4 * 5 = 120$$

$$\text{Fact}(n) = 1 * 2 * 3 * \dots * (n-1) * n$$

$$\text{Fact}(n) = \text{Fact}(n-1) * n$$

$$\text{Fact}(n) = \begin{cases} 1 & n=0 \\ \text{Fact}(n-1) * n & n>0 \end{cases}$$

```
int Fact(int n)
```

```
{
    IF(n==0)
        return 1;
```

```
    else
```

```
        return Fact(n-1) * n;
```

```
}
```

Exponent (m) ^{n}

$$2^5 = \underbrace{2 * 2 * 2 * 2 * 2}_{4 \text{ times}} * 2$$

$$m^n = m * m * m * \dots \text{For } n \text{ times}$$

$$\text{Pow}(m, n) = (m * m * m * \dots * m \text{ } n-1 \text{ times}) * m$$

$$\text{Pow}(m, n) = \text{Pow}(m, n-1) * m$$

$$\text{Pow}(m, n) = \begin{cases} 1 & n=0 \\ \text{Pow}(m, n-1) * m & n > 0 \end{cases}$$

$$\text{Pow}(2, 9)$$

int Pow(int m, int n)

|

$$\text{Pow}(2, 8) * 2$$

if (n == 0)

return 1;

|

$$\text{Pow}(2, 7) * 2$$

return Pow(m, n-1) * m;

|

$$\text{Pow}(2, 6) * 2$$

|

$$\text{Pow}(2, 5) * 2$$

|

$$\text{Pow}(2, 4) * 2$$

|

$$\text{Pow}(2, 3) * 2$$

|

$$\text{Pow}(2, 2) * 2 = 2^3$$

|

$$\text{Pow}(2, 1) * 2 = 2^2$$

|

$$\text{Pow}(2, 0) * 2 = 1 * 2$$

in, above way there is more call so consuming more memory as well as time, it can be reduced (no. of step) in following way

$$2^8 = (2^2)^4$$

$$= (2 * 2)^4$$

By Performing one multiplication
Power reduced by half

$$2^9 = 2 * (2^2)^4$$

Faster version

$$\text{Pow}(2, 4) = 2^4$$

$$2 * \text{Pow}(2^2 = 2^2, 4) = 2 * 2^8 = 2^9$$

$$\text{Pow}(2^2 * 2^2, 2)$$

$$\text{Pow}(2^4 * 2^4, 1) = 2^8$$

$$\text{Pow}(2^8 * 2^8 = 2^{16}, 0) = 2^8 * 2^8$$

int Pow(int m, int n)

if (n == 0)

return 1;

if (n % 2 == 0)

return Pow(m * m, n/2);

else

return m * Pow(m * m, (n+1)/2);

Taylor Series: e^x

$$e^x = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots \text{ } n \text{ terms}$$

↓
having 3 operation addition, power & factorial

$$\text{Sum}(n-1) + a$$

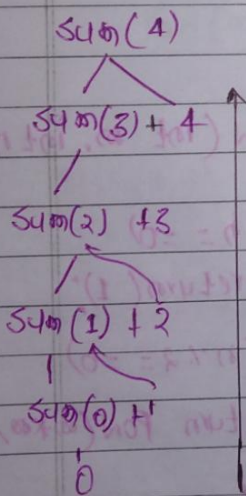
$$\text{Sum}(n) = 1 + 2 + 3 + \dots + n$$

$$\text{Fact}(n-1) * a$$

$$\text{Fact}(n) = 1 * 2 * 3 * \dots * n$$

$$\text{Pow}(x, n-1) * x$$

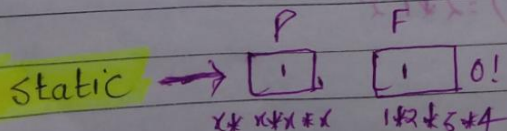
$$\text{Pow}(x, n) = \underbrace{x * x * x * \dots * x}_{n \text{ times}}$$



* These addition is done during returning time

For $n = 4$

$$e^x = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}$$



$$e(x, 4) = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}$$

$$e(x, 3) = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!}$$

$$e(x, 2) = 1 + \frac{x}{1} + \frac{x^2}{2!}$$

$$e(x, 1) = 1 + \frac{x}{1}$$

$$e(x, 0) = 1$$

$$P = P * x \quad F = F * 1 \quad 1 + P/F$$

these things
were performed
during returning
time.

```
int e (int x, int n)
```

```
{
    static int P=1, F=1;
```

```
    int r;
```

```
    if (n==0)
```

```
        return(1);
```

```
    else
```

```
{
    r = e(x, n-1);
```

```
    P = P * x;
```

```
    F = F * 1;
```

```
    return r + P/F;
```

* Performed
during
returning
x should be it
is being
Performed by
every call

```

→
#include <stdio.h>

double e (int x, int n)
{
    static double p=1, f=1;
    double r;

    if (n==0)
        return 1;

    r = e(x, n-1);
    p = p * x;
    f = f * n;
    return r + p/f;
}

int main()
{
    printf("%.1f\n", e(1, 5));
    return 0;
}
    
```

```

#include <stdio.h>

double e (int x, int n)
{
    double s=1;
    int i;
    double num=1;
    double den=1;

    for (i=1; i<=n; i++)
    {
        num *= x;
        den *= i;
        s += num/den;
    }

    return s;
}

int main()
{
    printf("%.1f\n", e(1, 5));
    return 0;
}
    
```

Taylor Series using Horner's Rule

$$e^x = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots + n \text{ terms}$$

$$0 \quad 0 \quad \frac{x+x}{1 \times 2} \quad \frac{x \times x \times x}{1 \times 2 \times 3} \quad \frac{3}{3}$$

$$4 \quad 6 \quad + 8 \quad + 10$$

no of multiplication 2

$$2 [1 + 2 + 3 + 4 + \dots]$$

Quadratic

$$\frac{2 \times (n)(n+1)}{2} = n(n+1) = O(n^2)$$

multiplication

Can we reduce no. of multiplication
↓

$$1 + \frac{x}{1} + \frac{x^2}{1*2} + \frac{x^3}{1*2*3} + \frac{x^4}{1*2*3*4}$$

$$1 + \frac{x}{1} \left[1 + \frac{x}{2} + \frac{x^2}{2*3} + \frac{x^3}{2*3*4} \right]$$

$$1 + \frac{x}{1} \left[1 + \frac{x}{2} \left[1 + \frac{x}{3} + \frac{x^2}{3*4} \right] \right]$$

$$\left[1 + \frac{x}{1} \left[1 + \frac{x}{2} \left[1 + \frac{x}{3} \left[1 + \frac{x}{4} \right] \right] \right] \right]$$

only 4 multiplication Now $O(n) \rightarrow \text{Linear}$

loop

```
int e (int x, int n)
```

```
{
```

```
    int s = 1;
```

```
    for ( ; n > 0; n--)
```

```
    {
```

```
        return 1 + x * s;
```

```
    }
```

```
    return s;
```

```
}
```

Recursion

```
int e (int x, int n)
```

```
{
```

```
    static int s = 1;
```

```
    if (n == 0)
```

```
        return s;
```

```
    s = 1 + x * s;
```

```
    return e(x, n-1);
```

```
}
```

H.W

Write sine series and cosine series

Fibonacci Series using Memoization

Fib(n)	0	1	1	2	3	5	8	13
n	0	1	2	3	4	5	6	7

$$Fib(n) = \begin{cases} 0 & n=0 \\ 1 & n=1 \\ Fib(n-2) + Fib(n-1) & n>1 \end{cases}$$

100P

int Fib(int n)

int t₀=0, t₁=1, s, i; — 1

IF (n <= 1) —

return n; — 1

For (i=2; i<=n; i++) — n

s = t₀ + t₁ — n-1

t₀ = t₁ — n-1

t₁ = s — n-1

return s; — $\frac{1}{4n}$

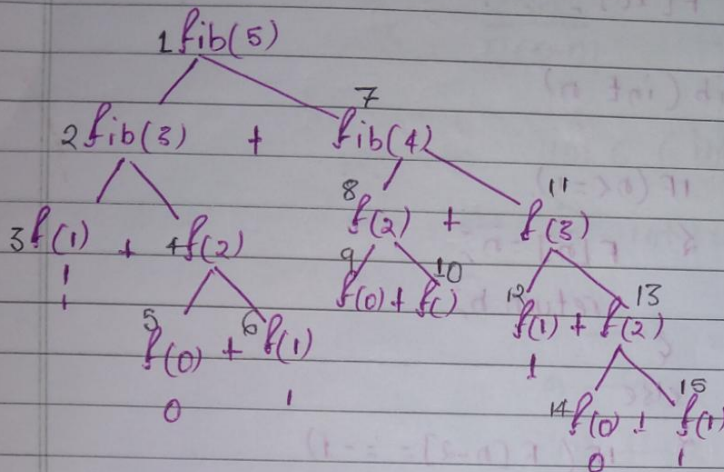
↓
For printing nth term O(n)

Excessive Recursion

IF it calls itself multiple time for the same parameter

Page No. 75

Date: / /



int fib(int n)

{

if (n <= 1)

return n;

return fib(n-2) + fib(n-1);

}

it is like tree recursion

Let 2, fib(n-1)

↓

$O(2^n)$

Call
F(5) = 15

F(4) = 9

F(3) = 5

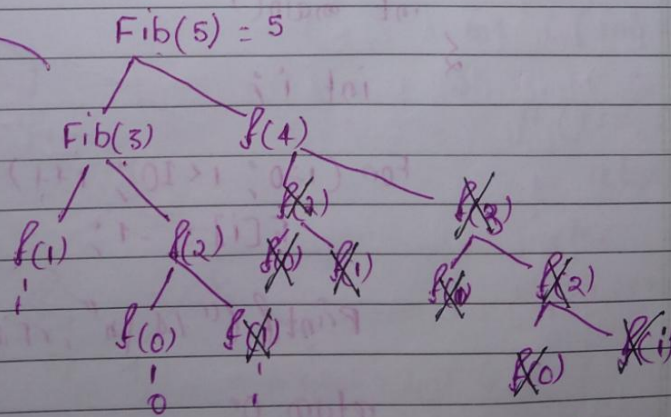
↓

to reduce order

As above F^n calling itself multiple time for same parameter we can store in static/global variable for that parameter

for 5 only six calls

$O(n)$



Memoization

F	0	1	1	2	3	5	8
	0	1	2	3	4	5	6

static or global variable initialised to 1

storing the result of function call so that they can be utilised when we need the same call is called Memoization

Using memoization

int F[10];

int fib(int n)

if (n <= 1)

F[n] = n;

return n;

else

if (F[n-2] == -1)

F[n-2] = fib(n-2);

if (F[n-1] == -1)

F[n-1] = fib(n-1);

F[n] = F[n-2] + F[n-1];

return F[n-2] + F[n-1];

int main()

int i;

for (i=0; i<10; i++)

F[i] = -1;

printf("1d\n", rFib(5));

return 0;

* Visualise
by tracking for some
value of n



nCr Using Recursion

$$nCr = \frac{n!}{r!(n-r)!}$$

int c (int n, int r)

{ int t1, t2, t3;

n — $t_1 = \text{fact}(n)$ factorial call

n — $t_2 = \text{fact}(r)$

n — $t_3 = \text{fact}(n-r)$

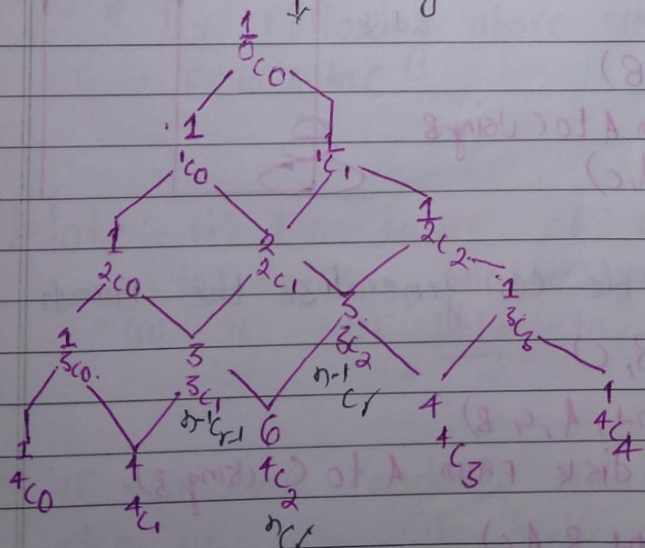
1 — return $t_1 / (t_2 * t_3)$

3n {

O(n)

↓
We can also write by using recursion

Pascal's Triangle



int c (int n, int r)

{ if (r == 0 || n == r)

return 1

else

return $c(n-1, r-1) + c(n-1, r)$

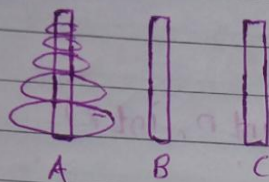
Every terms obtained by adding
Previous two

$$nCr = n-1Cr-1 + n-1Cr \quad \left[\begin{array}{ccc} 4C2 & = & 3C1 + 3C2 \\ 6 & = & 3 + 3 \end{array} \right]$$

Tower of Hanoi

Page No. 78

Date: / /



All disk must be transferred From A to C

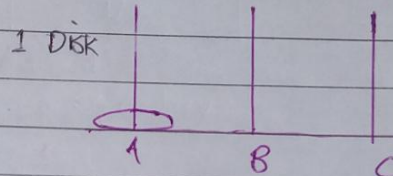
① Only one disk will be transferred one time

② No bigger disk will be kept on smaller disk

From using to

TOH(1, A, B, C)

Move disk From A to C using B

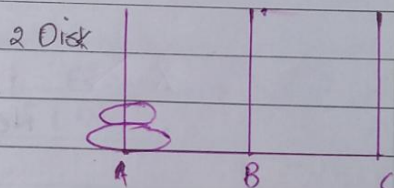


TOH(2, A, B, C)

1. TOH(1, A, C, B)

2. Move disk From A to C using B

3. TOH(1, B, A, C)

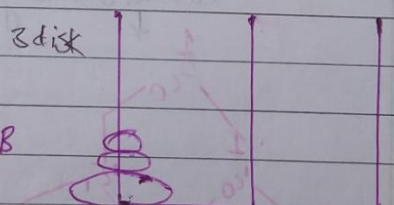


TOH(3, A, B, C)

1. TOH(2, A, C, B)

2. Move disk From A to C using B

3. TOH(2, B, A, C)



From above we can generalize the Formula

TOH(n, A, B, C)

1. TOH(n-1, A, C, B)

2. Move disk From A to C using B

3. TOH(n-1, B, A, C)

Generalized Formula

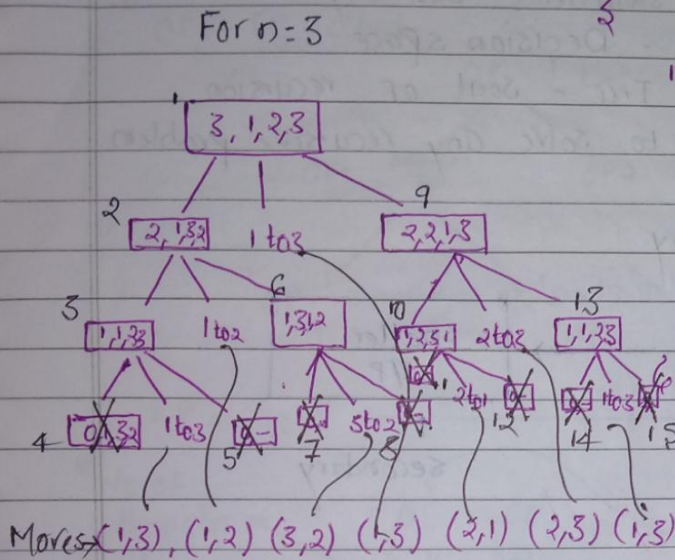
void TOH (int n, int A, int B, int C)
From using to

if (n > 0)

TOH (n-1, A, C, B)

printf ("From A to C", A);

TOH (n-1, B, A, C);



TOH (3, 1, 2, 3)
↑ ↑ ↑
A B C

Call

$$1 + 2 + 2^2 + 2^3 + \dots + 2^{n-1} = 2^n - 1 \quad n=3 \quad 15 \quad 1 + 2 + 2^2 + 2^3 = 2^4 - 1$$

$$= O(2^n) \quad n=2 \quad 7$$

★ By Following above Moves all disc will move From A to C

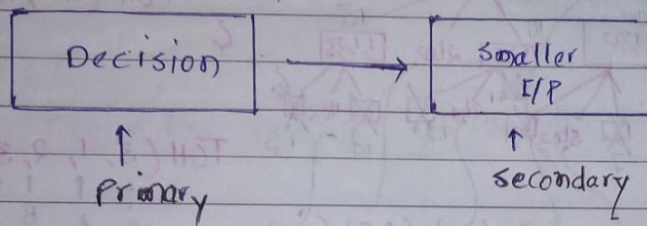
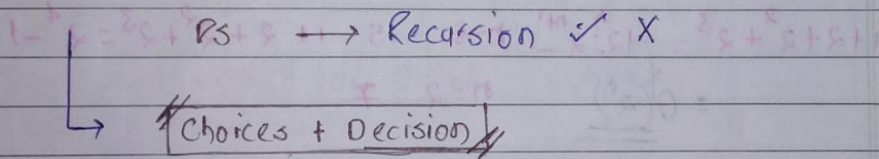
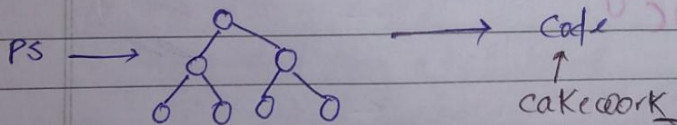
Note:- (i) This tower of Hanoi problem will help to solve another problem
(ii) It will help to think recursively

★ If you move randomly then you will make wrong step or you will make so many repeated step so it is better to think recursively

Flow:

1. Make I/P smaller? But why
2. Recursion - Decision space
3. Recursion Tree - soul of recursion
4. 2 step to solve any recursive problem

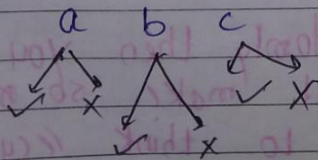
1. Make I/P smaller? why

2. Recursion - Decision space
identified3. Recursive Tree → loop

Example - subset problem

→ "abc" →

a	ab
b	bc
c	ac

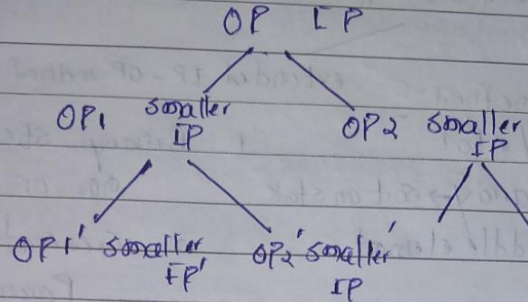


Recursion →

choices + Decision

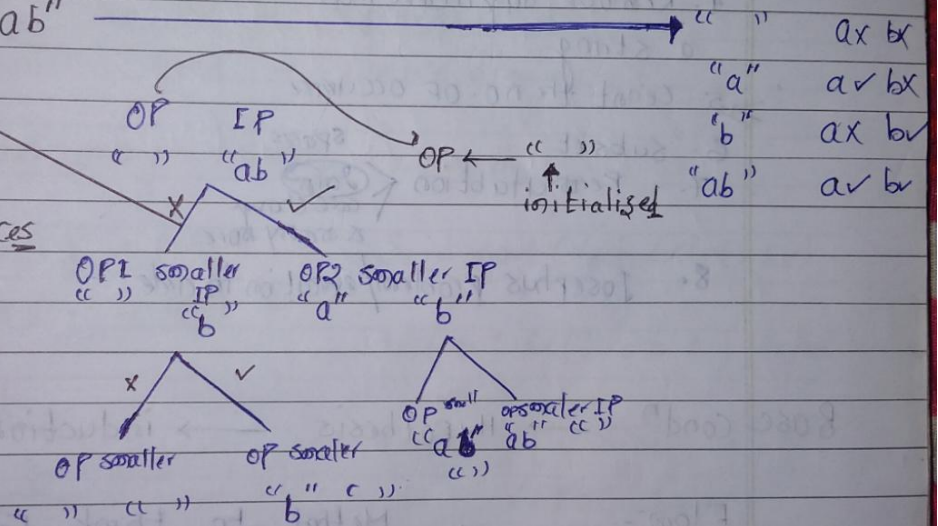
→ Recursion

"OP - IP Method"



IP = "ab"

*x No OF
Branches
↓ ways
No OF choices



1. Design a Recursive Tree
 2. Fuck the problem
- Case ← write the code

Recursion is Everywhere

Recursion Problem

IP - O/P method

extended IP - OP method

1. Print 1 to n / n to 1

2. Sort an array $\rightarrow$ sort on stack

3. Delete middle element in a stack

4. Remove duplicate from a string

5. Count the no. of occurrences

6. Subset

7. Permutation

spaces

Pair
case change
& many more

8. Josephus problem / execution in circle

1. Binary string $\bar{e}$

no. of 1's & no. of 0's

2. Generate Balanced Parenthesis

Base condⁿ $\rightarrow$ Hypothesis $\rightarrow$ induction

Flow:-

$\rightarrow$ Method to think for a recursive Problem

$\rightarrow$ why making I/P smaller is Pop/push

$\rightarrow$ Print 1 to N



Date

Page

83

4 Approaches

1) Recursive Tree - IP/OP method (Decision)

Making I/P smaller

2) BC - Induction - Hypothesis

3) choice diagram (DP)

simple $\rightarrow$ IBH

medium $\rightarrow$ Recursive Tree

Hard $\rightarrow$ choice diagram

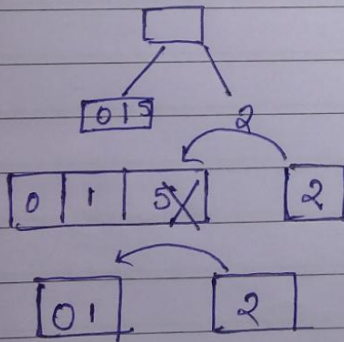
Q. Sort an array using recursion

sort (1 5 0) $\rightarrow$ 0 1 5

void sort (vector<int> &v)

sort (1 5 0) $\rightarrow$ [0 1 5]

if (v.size() == 1) return



[0 1 2]

void insert (vector<int> &v, int x)

if (v.size() == 0 || v[v.size()-1] < x)

v.push_back(x)

return

int x = v[v.size()-1]

v.pop_back()

insert (v, x)

v.push_back(x)

return



15100

Q. sort a stack using recursion

void sort(stack<int> &s)

if (s.size() <= 1) return;

int temp = s.top();

s.pop();

sort();

insert(s, temp);

void insert(stack<int> &s, int temp)

if (s.size() == 0 || s.top() <= temp)

s.push(temp);

return;

{

int val = s.top();

s.pop();

insert(s, temp);

s.push(val);

{

1. Pr

2. s

3. De

in

4. R

a

5. C

6. s

7.

8.

Cond

Flow

84



Date

Page

85



Date

Page

86

Recursion

1. P

2.

3. D

ir

4. K

a

5. C

6. i

7.

8.

e Cond

Floa

+ Approaches →

1) Recursive Tree - IP/OP method (Decision)

(Making IP smaller)

2) BC - Induction - Hypothesis

3) choice diagram (DP)

simple → IBH

medium → Recursive Tree

Hard → choice diagram

Q. Subset I

Given an integer array nums of unique elements return all possible subset (the Power set).

The solution must not contain duplicate subsets.
Return the solution in any order.

nums = [1, 2, 3]

o/p: [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]

I/P: nums = [0]

o/p: [[], [0]]

Follow up

** you have solved it by self

[Using Picking and Picking concept]

Q. T

Q. Subset II

nums that may contain duplicate element, return all possible subsets (the power set).

The solution set must not contain duplicate subsets.
Return the solution in any order

Rec

Input: nums = [1, 2, 2]
Output: [[], [1], [1, 2], [1, 2, 2], [2], [2, 2]]

Input: nums = [0]
O/P: [[0], []]

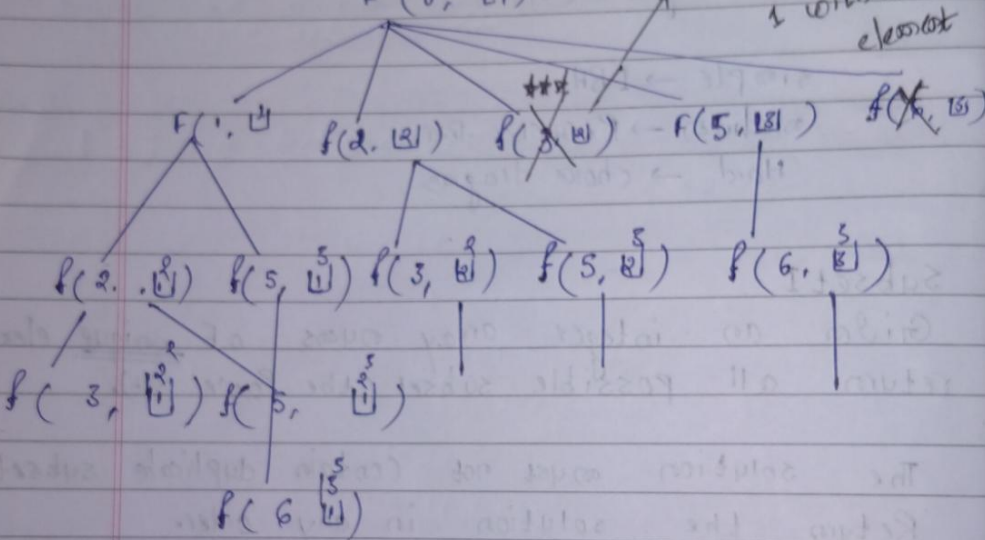
M-2 of
Joid

[1, 2, 2, 2, 3, 3]
0 1 2 3 4 5
[1], [1], [2], [3]
[1, 2], [1, 3], [2, 2], [2, 3], [3, 3]

List
F(0, Li)

Don't pick another
list of size
1 with 2 as
element

5
need of pop-
as m is not
to its parent
it is running
inside for



vector

Don't
Forget
to sort

Bas

F(ind, ds)

i = ind - (n-1)

avoid duplicate

f(i+1, ds)

↓

c

nums = [0]
[[0], [1]]

Take another
list of size
with 2 as
element

3 (10)

3
8

M-2 OF Take U Forward

using For

93

void Findsubset(int ind, vector<int> &nums, vector<int> &ds, vector<vector<int>> &res)

Need of Pop-back
as it is not return
to its parent rather
it is returning
inside for loop

ans.push_back(ds); → First time it will push empty ds

for (int i = ind; i < nums.size(); i++) {

if (i != ind & & nums[i] == nums[i-1]) continue;

ds.push_back(nums[i]);

Findsubset(nums[i], i+1, nums, ds, ans);

ds.pop_back();

Don't Forget
(you did mistake)

vector<vector<int>> subsetWithDup(vector<int> &nums)

vector<vector<int>> ans;

vector<int> ds;

sort(nums.begin(), nums.end());

Don't
Forget
to sort

Findsubsets(0, nums, ds, ans);

return ans;

[concept] Creating subset of firstly since 1, 2, 3, 4
x so on



Date

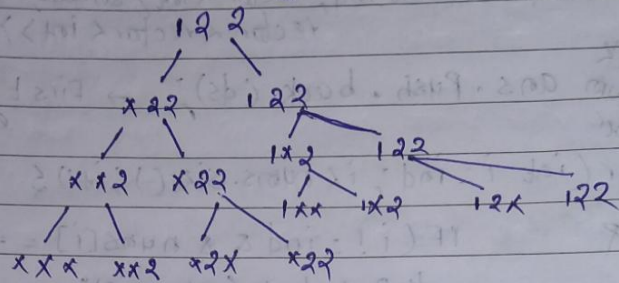
Page

94

Re

M-1 (your) → Picking & not picking concept)

Q. Co



void traverse (vector<int> &nums, int i, int n, vector<int> &v, set<vector<int> &S)

if (i == n)

{

traverse (nums, i+1, n, v, S);

v.push_back (nums[i]);

traverse (nums, i+1, n, v, S);

}

else if (i < n)

S.insert (v);

it will take care of duplicates

}

Bas

vector<vector<int>> subsetWithDup (vector<int> &nums)

{

vector<int> v;

vector<vector<int>> res;

Sort (nums.begin(), nums.end());

set<vector<int>> S;

int n = nums.size();

traverse (nums, 0, n, v, S);

set<vector<int>> :: iterator it;

it = S.begin();

while (it != S.end())

{ res.push_back (*it);

it++;

return res;

* Don't forget to sort, you did mistake while solving

R.

Concept)

Q. Combination Sum I

The same number may be chosen unlimited number of times.

Input: candidates = [2, 3, 6, 7], target = 7

Output: - [[2, 2, 3], [7]]

Input: candidates = [2, 3, 5], target = 8

Output: - [[2, 2, 2, 2], [2, 3, 3], [3, 5]]

void traverse (vector<int> & candidates, int i, int n, int sum, vector<int> & res, int target)

{

if (sum == target)

{ res.push_back(v);

}

else if (i < n && sum < target)

{

traverse (candidates, i+1, n, sum, v, res, target);

v.push_back (candidates[i]);

traverse (candidates, i, n, sum + candidates[i],

v, res, target);

}

}

vector<vector<int>> combinationSum (vector<int> & candidates, int target)

{

vector<vector<int>> res;

vector<int> v;

int sum = 0;

int n = candidates.size();

traverse (candidates, 0, n, sum, v, res, target);

}

OR



Date

Page

96

Q. Combination Sum II

Find all unique combinations in candidate where the candidate numbers sum to target

Each number in candidates may only be used once in combination:

Note: The solution set must not contain duplicate combinations

I/P: candidates = [10, 1, 2, 7, 6, 1, 5], target = 8

O/P: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

I/P: candidates = [2, 5, 2, 1, 2], target = 5

O/P: [[1, 2, 2], [5]]

void traverse (candidates, int ind, int sum, v, res)

```
if (sum == target)
{
    res.push_back(v);
    return;
}
```

```
for (int i = ind; i < n; i++)
```

```
if (i != ind && candidates[i] == candidates[i-1])
    continue;
```

```
if (candidates[i] > target)
    break;
```

```
sum += candidates[i];
```

```
v.push_back(candidates[i]);
```

```
traverse(candidates, i+1, n, sum, v, res, target);
```

```
sum -= candidates[i];
```

```
v.pop_back();
```

```
sort(candidates, candidates+n);
traverse();
```


date culture

used only

duplicate

[["a"]]

res)

Candidatos (i)

$a/a/b/b$

$a/b/b/b$

am I able to partition at 0 index & so on.

$f(s, 0)$

0 / []

$\frac{a}{a} \frac{b}{b}$

am I able to Partition at 0 index & so on)

$f(s, 0)$

looping

$(0, 0) \rightarrow f(s, 1)$

$(0, 1) \rightarrow f(s, 2)$

$(0, 2) \rightarrow f(s, 3)$

$(0, 3) \rightarrow f(s, 4)$

$(0, 4) \rightarrow f(s, 5)$

$(0, 5) \rightarrow f(s, 6)$

$(0, 6) \rightarrow f(s, 7)$

$(0, 7) \rightarrow f(s, 8)$

$(0, 8) \rightarrow f(s, 9)$

$(0, 9) \rightarrow f(s, 10)$

$(0, 10) \rightarrow f(s, 11)$

$(0, 11) \rightarrow f(s, 12)$

$(0, 12) \rightarrow f(s, 13)$

$(0, 13) \rightarrow f(s, 14)$

$(0, 14) \rightarrow f(s, 15)$

$(0, 15) \rightarrow f(s, 16)$

$(0, 16) \rightarrow f(s, 17)$

$(0, 17) \rightarrow f(s, 18)$

$(0, 18) \rightarrow f(s, 19)$

$(0, 19) \rightarrow f(s, 20)$

$(0, 20) \rightarrow f(s, 21)$

$(0, 21) \rightarrow f(s, 22)$

$(0, 22) \rightarrow f(s, 23)$

$(0, 23) \rightarrow f(s, 24)$

$(0, 24) \rightarrow f(s, 25)$

$(0, 25) \rightarrow f(s, 26)$

$(0, 26) \rightarrow f(s, 27)$

$(0, 27) \rightarrow f(s, 28)$

$(0, 28) \rightarrow f(s, 29)$

$(0, 29) \rightarrow f(s, 30)$

$(0, 30) \rightarrow f(s, 31)$

$(0, 31) \rightarrow f(s, 32)$

$(0, 32) \rightarrow f(s, 33)$

$(0, 33) \rightarrow f(s, 34)$

$(0, 34) \rightarrow f(s, 35)$

$(0, 35) \rightarrow f(s, 36)$

$(0, 36) \rightarrow f(s, 37)$

$(0, 37) \rightarrow f(s, 38)$

$(0, 38) \rightarrow f(s, 39)$

$(0, 39) \rightarrow f(s, 40)$

$(0, 40) \rightarrow f(s, 41)$

$(0, 41) \rightarrow f(s, 42)$

$(0, 42) \rightarrow f(s, 43)$

$(0, 43) \rightarrow f(s, 44)$

$(0, 44) \rightarrow f(s, 45)$

$(0, 45) \rightarrow f(s, 46)$

$(0, 46) \rightarrow f(s, 47)$

$(0, 47) \rightarrow f(s, 48)$

$(0, 48) \rightarrow f(s, 49)$

$(0, 49) \rightarrow f(s, 50)$

$(0, 50) \rightarrow f(s, 51)$

$(0, 51) \rightarrow f(s, 52)$

$(0, 52) \rightarrow f(s, 53)$

$(0, 53) \rightarrow f(s, 54)$

$(0, 54) \rightarrow f(s, 55)$

$(0, 55) \rightarrow f(s, 56)$

$(0, 56) \rightarrow f(s, 57)$

$(0, 57) \rightarrow f(s, 58)$

$(0, 58) \rightarrow f(s, 59)$

$(0, 59) \rightarrow f(s, 60)$

$(0, 60) \rightarrow f(s, 61)$

$(0, 61) \rightarrow f(s, 62)$

$(0, 62) \rightarrow f(s, 63)$

$(0, 63) \rightarrow f(s, 64)$

$(0, 64) \rightarrow f(s, 65)$

$(0, 65) \rightarrow f(s, 66)$

$(0, 66) \rightarrow f(s, 67)$

$(0, 67) \rightarrow f(s, 68)$

$(0, 68) \rightarrow f(s, 69)$

$(0, 69) \rightarrow f(s, 70)$

$(0, 70) \rightarrow f(s, 71)$

$(0, 71) \rightarrow f(s, 72)$

$(0, 72) \rightarrow f(s, 73)$

$(0, 73) \rightarrow f(s, 74)$

$(0, 74) \rightarrow f(s, 75)$

$(0, 75) \rightarrow f(s, 76)$

$(0, 76) \rightarrow f(s, 77)$

$(0, 77) \rightarrow f(s, 78)$

$(0, 78) \rightarrow f(s, 79)$

$(0, 79) \rightarrow f(s, 80)$

$(0, 80) \rightarrow f(s, 81)$

$(0, 81) \rightarrow f(s, 82)$

$(0, 82) \rightarrow f(s, 83)$

$(0, 83) \rightarrow f(s, 84)$

$(0, 84) \rightarrow f(s, 85)$

$(0, 85) \rightarrow f(s, 86)$

$(0, 86) \rightarrow f(s, 87)$

$(0, 87) \rightarrow f(s, 88)$

$(0, 88) \rightarrow f(s, 89)$

$(0, 89) \rightarrow f(s, 90)$

$(0, 90) \rightarrow f(s, 91)$

$(0, 91) \rightarrow f(s, 92)$

$(0, 92) \rightarrow f(s, 93)$

$(0, 93) \rightarrow f(s, 94)$

$(0, 94) \rightarrow f(s, 95)$

$(0, 95) \rightarrow f(s, 96)$

$(0, 96) \rightarrow f(s, 97)$

$(0, 97) \rightarrow f(s, 98)$

$(0, 98) \rightarrow f(s, 99)$

$(0, 99) \rightarrow f(s, 100)$

$(0, 100) \rightarrow f(s, 101)$

$(0, 101) \rightarrow f(s, 102)$

$(0, 102) \rightarrow f(s, 103)$

$(0, 103) \rightarrow f(s, 104)$

$(0, 104) \rightarrow f(s, 105)$

$(0, 105) \rightarrow f(s, 106)$

$(0, 106) \rightarrow f(s, 107)$

$(0, 107) \rightarrow f(s, 108)$

$(0, 108) \rightarrow f(s, 109)$

$(0, 109) \rightarrow f(s, 110)$

$(0, 110) \rightarrow f(s, 111)$

$(0, 111) \rightarrow f(s, 112)$

$(0, 112) \rightarrow f(s, 113)$

$(0, 113) \rightarrow f(s, 114)$

$(0, 114) \rightarrow f(s, 115)$

$(0, 115) \rightarrow f(s, 116)$

$(0, 116) \rightarrow f(s, 117)$

$(0, 117) \rightarrow f(s, 118)$

$(0, 118) \rightarrow f(s, 119)$

$(0, 119) \rightarrow f(s, 120)$

$(0, 120) \rightarrow f(s, 121)$

$(0, 121) \rightarrow f(s, 122)$

$(0, 122) \rightarrow f(s, 123)$

$(0, 123) \rightarrow f(s, 124)$

$(0, 124) \rightarrow f(s, 125)$

$(0, 125) \rightarrow f(s, 126)$

$(0, 126) \rightarrow f(s, 127)$

$(0, 127) \rightarrow f(s, 128)$

$(0, 128) \rightarrow f(s, 129)$

$(0, 129) \rightarrow f(s, 130)$

$(0, 130) \rightarrow f(s, 131)$

$(0, 131) \rightarrow f(s, 132)$

$(0, 132) \rightarrow f(s, 133)$

$(0, 133) \rightarrow f(s, 134)$

$(0, 134) \rightarrow f(s, 135)$

$(0, 135) \rightarrow f(s, 136)$

$(0, 136) \rightarrow f(s, 137)$

$(0, 137) \rightarrow f(s, 138)$

$(0, 138) \rightarrow f(s, 139)$

$(0, 139) \rightarrow f(s, 140)$

$(0, 140) \rightarrow f(s, 141)$

$(0, 141) \rightarrow f(s, 142)$

$(0, 142) \rightarrow f(s, 143)$

$(0, 143) \rightarrow f(s, 144)$

$(0, 144) \rightarrow f(s, 145)$

$(0, 145) \rightarrow f(s, 146)$

$(0, 146) \rightarrow f(s, 147)$

$(0, 147) \rightarrow f(s, 148)$

$(0, 148) \rightarrow f(s, 149)$

$(0, 149) \rightarrow f(s, 150)$

$(0, 150) \rightarrow f(s, 151)$

$(0, 151) \rightarrow f(s, 152)$

$(0, 152) \rightarrow f(s, 153)$

$(0, 153) \rightarrow f(s, 154)$

$(0, 154) \rightarrow f(s, 155)$

$(0, 155) \rightarrow f(s, 156)$

$(0, 156) \rightarrow f(s, 157)$

$(0, 157) \rightarrow f(s, 158)$

$(0, 158) \rightarrow f(s, 159)$

$(0, 159) \rightarrow f(s, 160)$

$(0, 160) \rightarrow f(s, 161)$

$(0, 161) \rightarrow f(s, 162)$

$(0, 162) \rightarrow f(s, 163)$

$(0, 163) \rightarrow f(s, 164)$

$(0, 164) \rightarrow f(s, 165)$

$(0, 165) \rightarrow f(s, 166)$

$(0, 166) \rightarrow f(s, 167)$

$(0, 167) \rightarrow f(s, 168)$

$(0, 168) \rightarrow f(s, 169)$

$(0, 169) \rightarrow f(s, 170)$

$(0, 170) \rightarrow f(s, 171)$

<
$$F(s, ind)$$

loop ind $\rightarrow$ 9-

(if ind till is paleo
-drome then only
we can call
next cases



Date

Page

98

vector<vector<string>> Partition(string s)

vector<string> Path;

Func(0, s, Path, res);

return res;

}

void Func(int index, string s, vector<string> & Path,

vector<vector<string>> & res) {

if (index == s.size()) {

res.push_back(Path);

return;

}

for (int i = index; i < s.size(); i++) {

if (isPalindrome(s, index, i)) {

Path.push_back(s.substr(index, i - index + 1));

start, number

Func(i+1, s, Path, res);

Path.pop_back();

}

}

bool isPalindrome(string s, int start, int end)

while (start < end) {

if (s[start++] != s[end--])

return false;

}

return true;

}

Hard

Q. The set
Permuta
By 1
Order

Give

Input:
O/P:-

Recur

0

Permut
24
=

Hard

Q. The set $[1, 2, 3, \dots, n]$ contains a total of $n!$ unique permutations.

By listing and labelling all of the permutations in order, we get the following sequence for $n=3$:

1. "123"
2. "132"
3. "213"
4. "231"
5. "312"
6. "321"

Given n and k , return the k^{th} permutation sequence.

Input: $n=3, k=3$

O/P: "213"

Input: $n=4, k=9$

O/P: "2314"

$n=4, k=17$

Brute Force $\rightarrow$ Generate all permutation & store in ds.

Recursion

$O(n!)$ & OF sorting

then sort it & return k^{th}

Optimal solution

Permutation
24

$n=4, k=17$
1, 2, 3, 4
0, 1, 2, 3

$16/6 = 2$
 $16\%6 = 4$
 $1 + (2, 3, 4) \rightarrow 6$ (0-5)
 $2 + (1, 3, 4) \rightarrow 6$ (6-11)
 $3 + (1, 2, 4) \rightarrow 6$ (12-17)
 $4 + (1, 2, 3) \rightarrow 6$ (18-23)

24 unique Permutation

Now we have to Find 4th sequence out of 6

1, 2, 3, 4, $k=4$



$$2! = 2$$

$$1, 2, 4 \quad K=4 \quad 0 \quad 1 \quad 2+3 \quad | \quad 2 \quad (0-1)$$

$$1/2 = 2 \quad K=1+1-2=0 \quad 1 \quad 2 \quad 1+2 \quad | \quad 2 \quad (2-3)$$

$$2 \rightarrow 1 \quad 1, 2 \quad | \quad 2 \quad (4-5)$$

$$6$$

$$\frac{3}{1} + \frac{2}{1}$$

$$1, 2 \quad K=0/1=0, \quad K=0+0=0$$

$$2! = 2 \quad @ \quad 1 \quad + \quad 2 \quad | \quad 1$$

$$1 \quad 2 \quad + \quad 1 \quad | \quad 1$$

$$2$$

$$2 \quad 2 \quad K=0$$

$$\frac{3}{1} + \frac{2}{1} \quad 2$$

For erasing in sector

$$T.C \rightarrow O(N) \times O(N) = O(N^2)$$

Soid

String getPermutation (int n, int k) {
 int Fact = 1;

 vector<int> numbers;

 for (int i = 1; i <= n; i++)

 Fact = Fact * i;

 numbers.push_back(i);

 numbers.push_back(n);

 string ans = "";

 K = K - 1;

while (true) {
 ans = ans + to-string

 (numbers[k/Fact]);

 numbers.erase(numbers.begin() + k/Fact);

 if (numbers.size() == 0)
 break;

 K = K % Fact;

 Fact = Fact / numbers.size();

return ans;

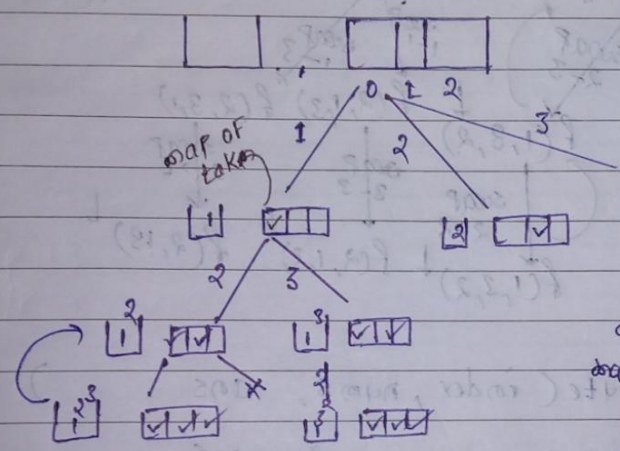
24 | 2 (0-1)
24 | 2 (2-3)
24 | 2 (4-5)
6

(#) Permutations

nums = [1, 2, 3]
O/Ps: [1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]

Approach - 1 [Using extra space] Pick & not pick

[1, 2, 3] n = 3, n! = 3! = 6
0 1 2



TC → O(n!)
SC → O(n), -map

vector

f(ds, map)
ds.add(arr)
map[i]++
loop(0 - n-1)
if (i is ! in map)
f(ds, map)

void recurPermute(vector<int> &ds, vector<int> &nums, vector<vector<int>> &ans, int freq[]) {

if (ds.size() == nums.size()) {
ans.push_back(ds);
return;

for (int i = 0; i < nums.size(); i++) {

if (!freq[i]) {
ds.push_back(nums[i]);
freq[i] = 1;
recurPermute(ds, nums, ans, freq);
freq[i] = 0;
ds.pop_back();

}
}
vector<vector<int>> permute(vector<int> &nums) {

recurPermute(ds, nums, ans, freq);
return ans;

}

Approach-2:- [without using extra space]

try every one
with index

 $[1, 2, 3]$
$$n = 3$$

~~SWAP 1 $\rightarrow$ 3~~

$$f(3, 2, 1)$$
 $\Gamma_C \rightarrow$

123

132

2, 3

2 3 1

3 2 1

३ १ २

```
recurPermute(index, nums, ans)
```

★ Pass index here
note (i), you
have done
mistake

if (index == max_size()) {

ans. push-back (nyons)

② $\text{recurPermut}(\text{index}+1, \text{nums}, \text{ans})$

$$\text{swap}(\text{nums}[\text{index}], \text{nums}[\text{i}]) \quad (6)$$

```
for (int i = index; i < vns.size(); i++)
```

~~swap(a4, a5, s[index], a4, s[i])~~

return ans;

6

by mistake
you written
48 & 4000